EX07 - Taylor for e^x: state, factorial and symbolic expression

Why the loop starts at n = 0, how n! is maintained without recomputing it, and how to display the series in the mathematical order we build it.

---

1. The mathematical idea

We want to build the Maclaurin approximation of the exponential. If we truncate at order N, the expression is:

$$e^x \approx \sum_{n=0}^{N} x^n / n!$$

The key point is that the code should mirror that structure: for each n, compute one term x^n/n! and add it to the series accumulator.

```
ex = sp.Integer(0)
fact = 1
terms = []

for n in range(0, N + 1):

    term = x**n / fact

    ex += term
    terms.append(term)

    expression_to_show = " + ".join(str(term) for term in terms)

    print(f"n = {n} | ex = {expression_to_show}")

    fact *= n + 1
```

> Read it in one sentence
>
> On each pass we use the factorial required by the current term, add that term to the series, and at the end prepare the factorial needed by the next pass.

2. What each variable represents

Variable	Initial	Role	Interpretation
ex	sp.Integer(0)	Symbolic series accumulator	Stores 1, 1+x, 1+x+x^2/2, ...
fact	1	Factorial required by current term	At the start of iteration n it must equal n!
terms	[]	List used only for display	Stores terms in the order in which we generate them
n	0..N	Counter / term order	Selects x^n/n!
term	x**n/fact	Current term	Contribution added during that iteration

Why `ex = sp.Integer(0)`

The mathematical value is still zero, but it is now a SymPy object. This ensures that the accumulator belongs to the symbolic world from the start and supports operations such as `subs()`. With this loop, which always includes n=0, it is not strictly mandatory because adding the first term would promote the expression to SymPy; nevertheless, it is a cleaner and more robust initialization.

> Mathematical value vs computational type
>
> `0` and `sp.Integer(0)` represent the same number, but they are not the same kind of object. The latter knows SymPy symbolic operations.

3. The key: fact always represents n!

This is the most important part of the algorithm. At the start of every iteration we want the following rule to hold:

fact = n! (when entering iteration n)

This property is called a loop invariant: a relationship that remains true whenever a new pass begins. Thanks to it, we do not need to recompute n! from scratch.

n	fact on entry	term	fact after update
0	1 = 0!	x^0 / 1 = 1	1 * (0+1) = 1 = 1!
1	1 = 1!	x / 1 = x	1 * (1+1) = 2 = 2!
2	2 = 2!	x^2 / 2	2 * (2+1) = 6 = 3!
3	6 = 3!	x^3 / 6	6 * (3+1) = 24 = 4!
4	24 = 4!	x^4 / 24	24 * (4+1) = 120 = 5!

> Why the update goes at the end
>
> If we executed `fact *= n + 1` before computing `term`, then at n=1 we would already be using 2! instead of 1!. Instruction order is part of the algorithm.

4. Why n = 0 fits inside the same loop

The series truly starts at n=0. That first term does not need an exception:

$$x^0 / 0! = 1 / 1 = 1$$

If we initialize `fact = 1` and start with `range(0, N + 1)`, the first iteration generates exactly that term. Therefore we can remove an `if N > 0` branch and keep the whole series in one flow.

> Simplification principle
>
> If the general algorithm already handles a special case correctly, there is no need to create a dedicated branch for it.

Comparison with the previous version

Before

```
ex = 1
fact = 1

for n in range(1, N + 1):
    fact *= n
    ex += x**n / fact

if N > 0:
    ex = ex.subs(x, xp)
```

Now

```
ex = sp.Integer(0)
fact = 1

for n in range(0, N + 1):
    term = x**n / fact
    ex += term
    fact *= n + 1

ex_numeric = ex.subs(x, xp)
```

5. terms and join(): controlling display only

SymPy reorganizes sums into a canonical algebraic order. Therefore it may display:

```
x**3/6 + x**2/2 + x + 1
```

even though we generated the terms as:

```
1 + x + x**2/2 + x**3/6
```

Both expressions are mathematically identical. If we want to see the pedagogical order in which the series is built, we use a separate list:

```
terms.append(term)
expression_to_show = " + ".join(str(term) for term in terms)
```

`terms.append(term)` stores the current symbolic object at the end of the list.

`str(term) for term in terms` walks through the list and converts each term to text.

`" + ".join(...)` concatenates those strings and inserts the separator `+` between them.

> Important
>
> `terms` and `expression_to_show` do not take part in the mathematical calculation. They are only a presentation layer so that we can read the series in the order we want.

6. Full execution for N = 4

n	fact used	term	ex after add	fact prepared
0	1	1	1	1!
1	1	x	1 + x	2!
2	2	x^2/2	1 + x + x^2/2	3!
3	6	x^3/6	1 + x + x^2/2 + x^3/6	4!
4	24	x^4/24	1 + x + x^2/2 + x^3/6 + x^4/24	5!

The conceptual sequence on every pass is:

1) use current state -> 2) compute term -> 3) accumulate it -> 4) display it -> 5) prepare the next iteration state

7. Recommended pattern for your EX07

```
from math import exp
import sympy as sp

N = 5
x = sp.symbols("x")
xp = 0.5

ex_symbolic = sp.Integer(0)
fact = 1
terms = []

for n in range(0, N + 1):
    term = x**n / fact
    ex_symbolic += term
    terms.append(term)

    expression_to_show = " + ".join(str(t) for t in terms)
    print(f"n = {n} | ex = {expression_to_show}")

    fact *= n + 1

ex_numeric = ex_symbolic.subs(x, xp)
analytical_result = exp(xp)
```

Keeping `ex_symbolic` separate from `ex_numeric` preserves both the formula and its evaluation. This will be especially useful when we start plotting curves, comparing errors, and turning the code into reusable functions.

> Learning tracker
>
> New concepts: SymPy symbolic object, loop invariant, current state vs future state, update order, display separated from calculation, lists as history, generator expression, and `join()`.

8. Common mistakes to avoid

Mistake	Why it matters
Updating fact too early	You would use (n+1)! in the nth term.
Calling math.factorial(n) every pass	It works, but it bypasses the accumulator exercise and recomputes work we already know.
Mixing symbolic ex with numeric ex	An expression in x and its value at xp are conceptually different objects.
Assuming SymPy display order equals algorithm order	SymPy may reorder a sum algebraically without changing its value.